DVWA Security Level Comparison Project Report

Title: DVWA Security Level Comparison Project

Name: Rahul Gandham

Date: 30-09-2025

Table of Contents :

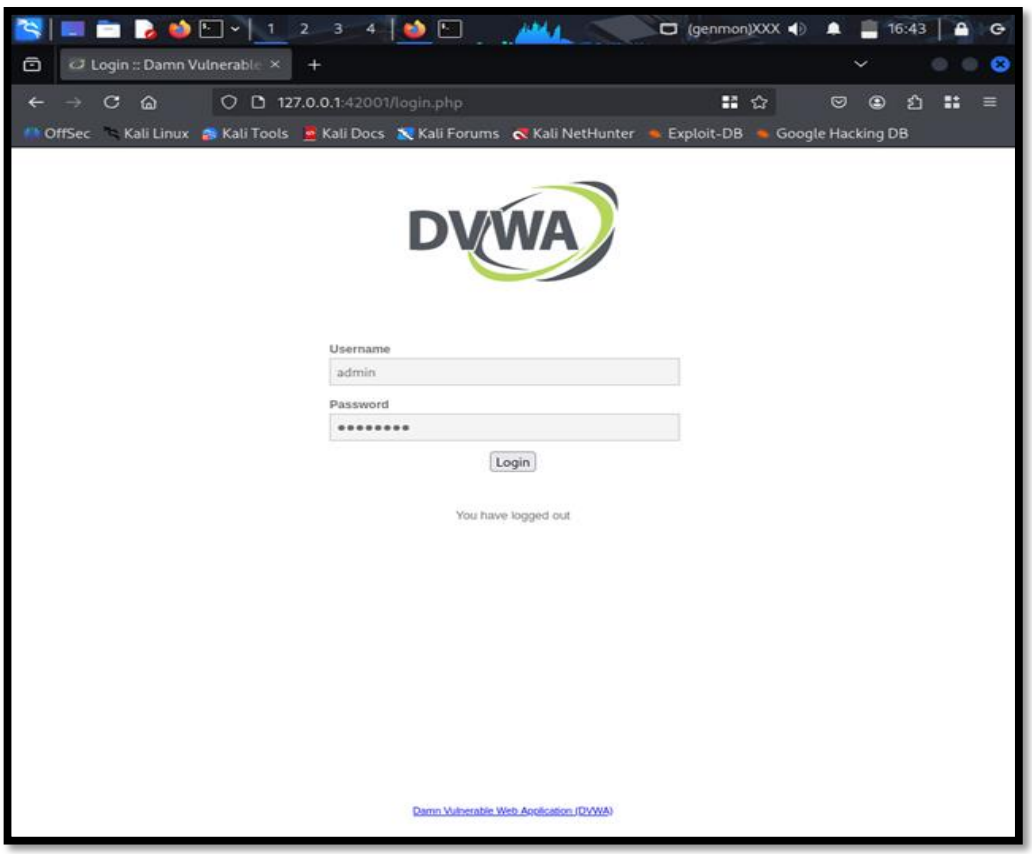
1. Introduction

The Damn Vulnerable Web Application (DVWA) is a deliberately insecure web application designed to educate and train security professionals in identifying and exploiting common web vulnerabilities safely. This project focuses on evaluating DVWA's behavior at four security levels—Low, Medium, High, and Impossible—through the testing of two common vulnerabilities: SQL Injection (SQLi) and Reflected Cross-Site Scripting (XSS).

This analysis highlights the evolution of security defenses implemented by DVWA to mitigate attacks, illustrating real-world defensive programming practices and security controls. Understanding these defense mechanisms is critical for anyone aspiring to work in cybersecurity or web application security.

2. Environment Setup and Tools Used

- DVWA hosted on local VM with Apache and MySQL.
- Testing controlled in offline environment ensuring safety.Tools used for manual testing included browser input forms and intercepting proxies like Burp Suite for request manipulation.



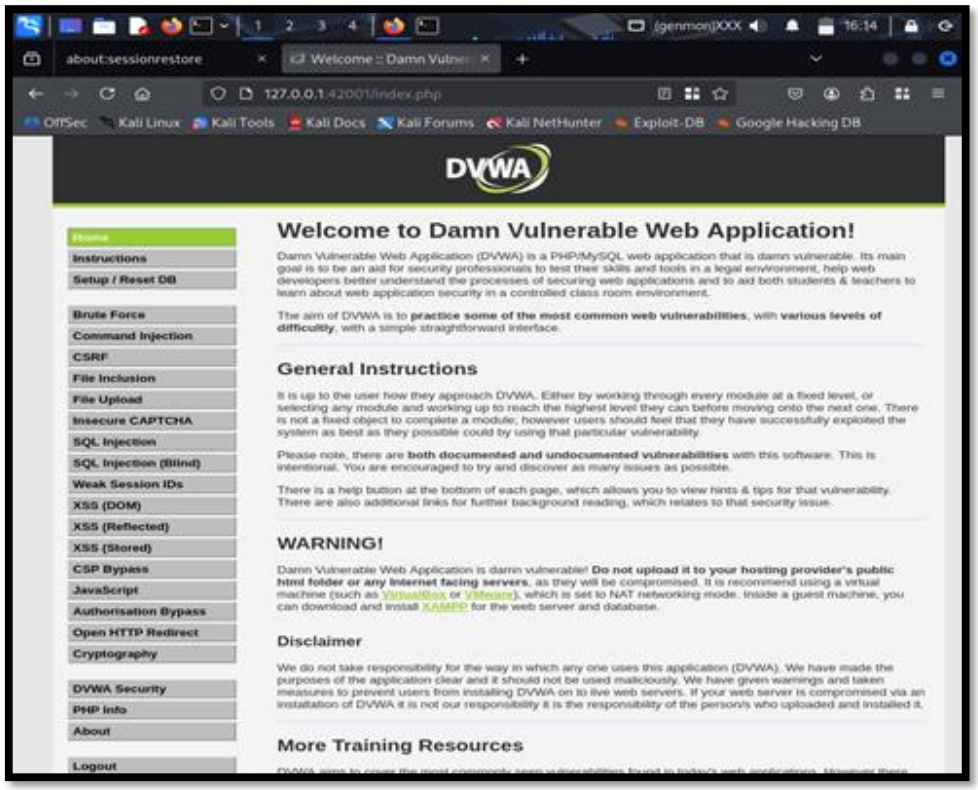
Screenshot 1:

- DVWA login page and configuration dashboard.
  Description: Demonstrates the operational environment and successful setup of DVWA.

3. Vulnerabilities Selected

Two vulnerabilities were chosen to demonstrate attack mechanisms and defense strategies:

- **SQL Injection (SQLi):** Inject malicious SQL queries via input parameters to extract unauthorized database information.
- **Reflected Cross-Site Scripting (XSS):** Inject malicious JavaScript payloads via URL or form inputs which get reflected and executed in the victim's browser.



Screenshot 2:

- DVWA interface showing 'SQL Injection' and 'XSS (Reflected)' module selection.
  Description: Verifies the project scope and features selected for deep analysis.

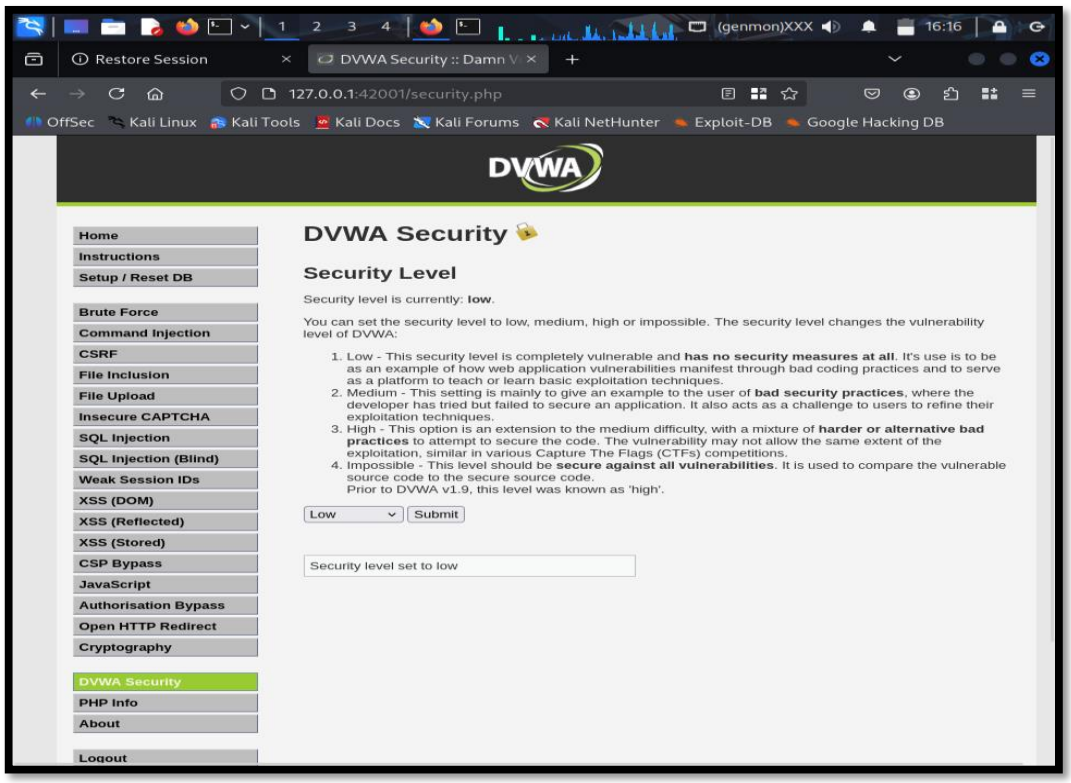
4. Detailed Testing Procedure and Observations

4.1 SQL Injection

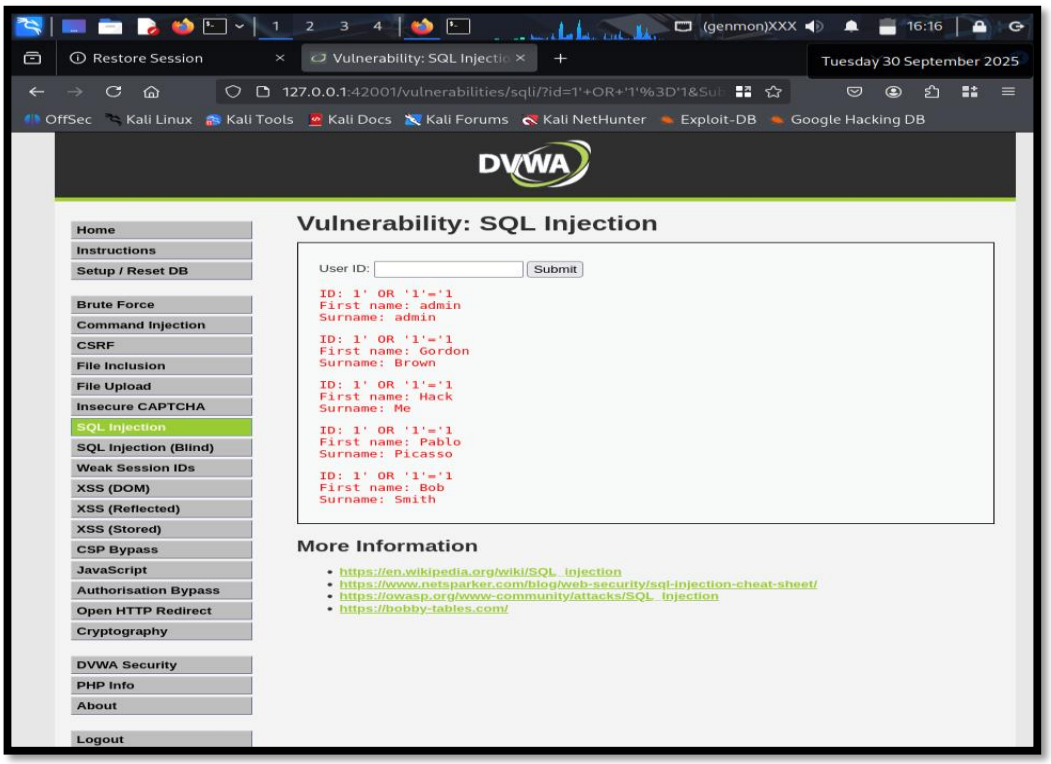
SQL Injection was tested across security levels by submitting increasingly filtered SQL payloads and observing responses.

Low Security

- **Payload:**
  `1' OR '1'='1`
- **Outcome:** Full database data leaked, confirming no filtering or input sanitization.
- **Explanation:** The application concatenates user input directly into SQL queries, allowing logical tautologies to bypass authentication or retrieve unintended data.



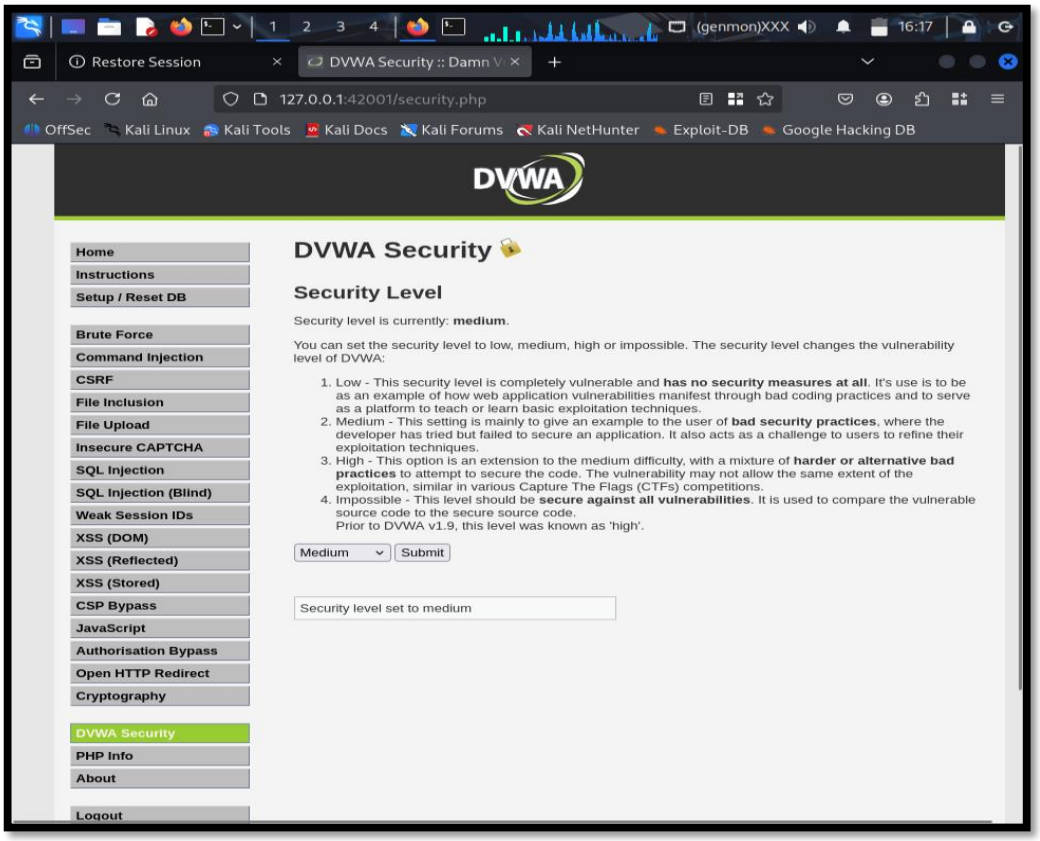
Screenshot 3: Security level set to " Low "



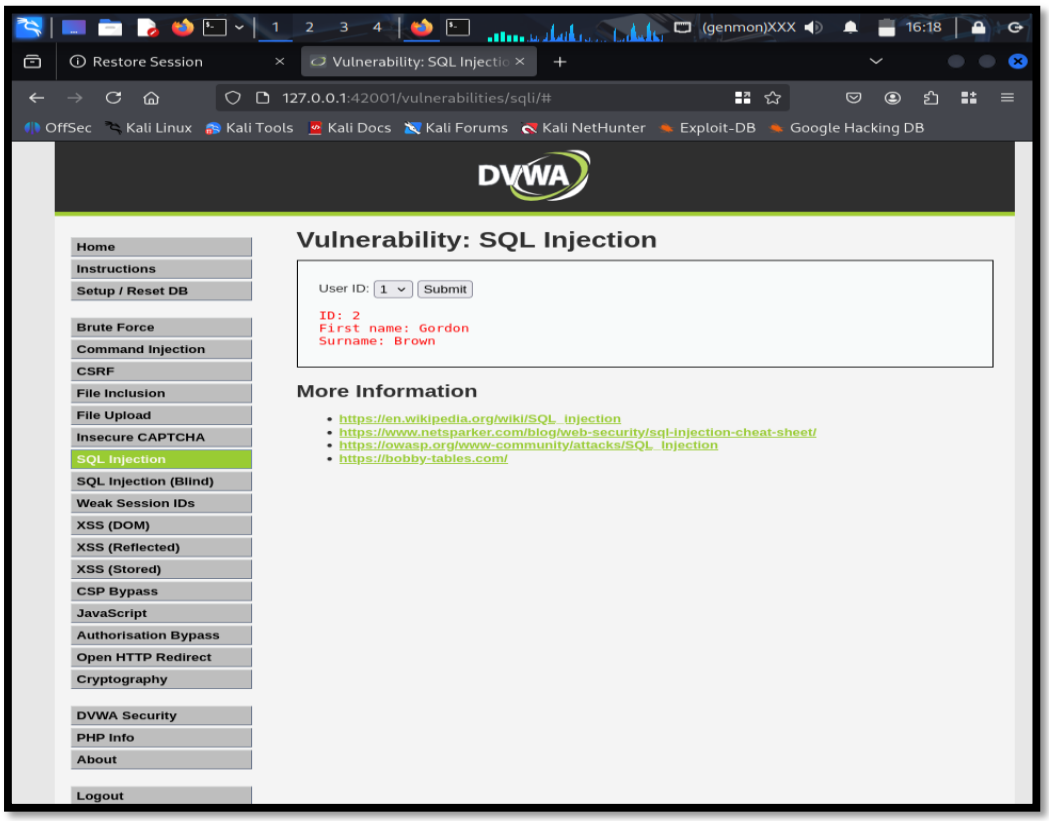
- **Screenshot 4:** Showing input form with payload and resulting page displaying leaked data.
  Description: Clear evidence of SQL Injection with no protection.

Medium Security

- **Payload:**
  `1 UNION SELECT user,password FROM users#`
- **Outcome:** Partial filtering but still vulnerable, application leaks data conditionally.
- **Explanation:** Moderate input validation implemented; however, strict parameterization or prepared statements are not used, allowing some bypasses for union-based SQLi.



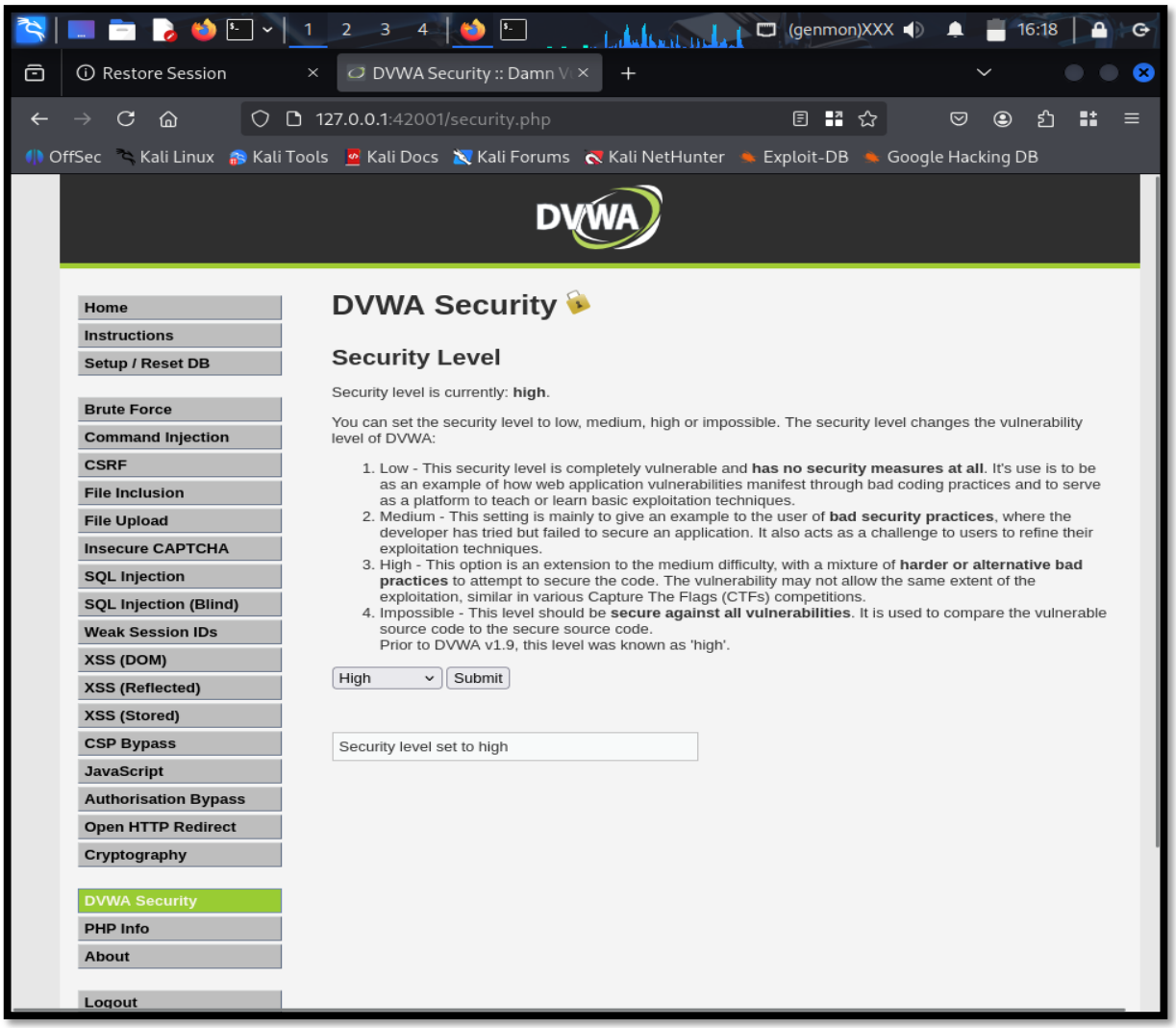
Screenshot 5: Security Level set to Medium



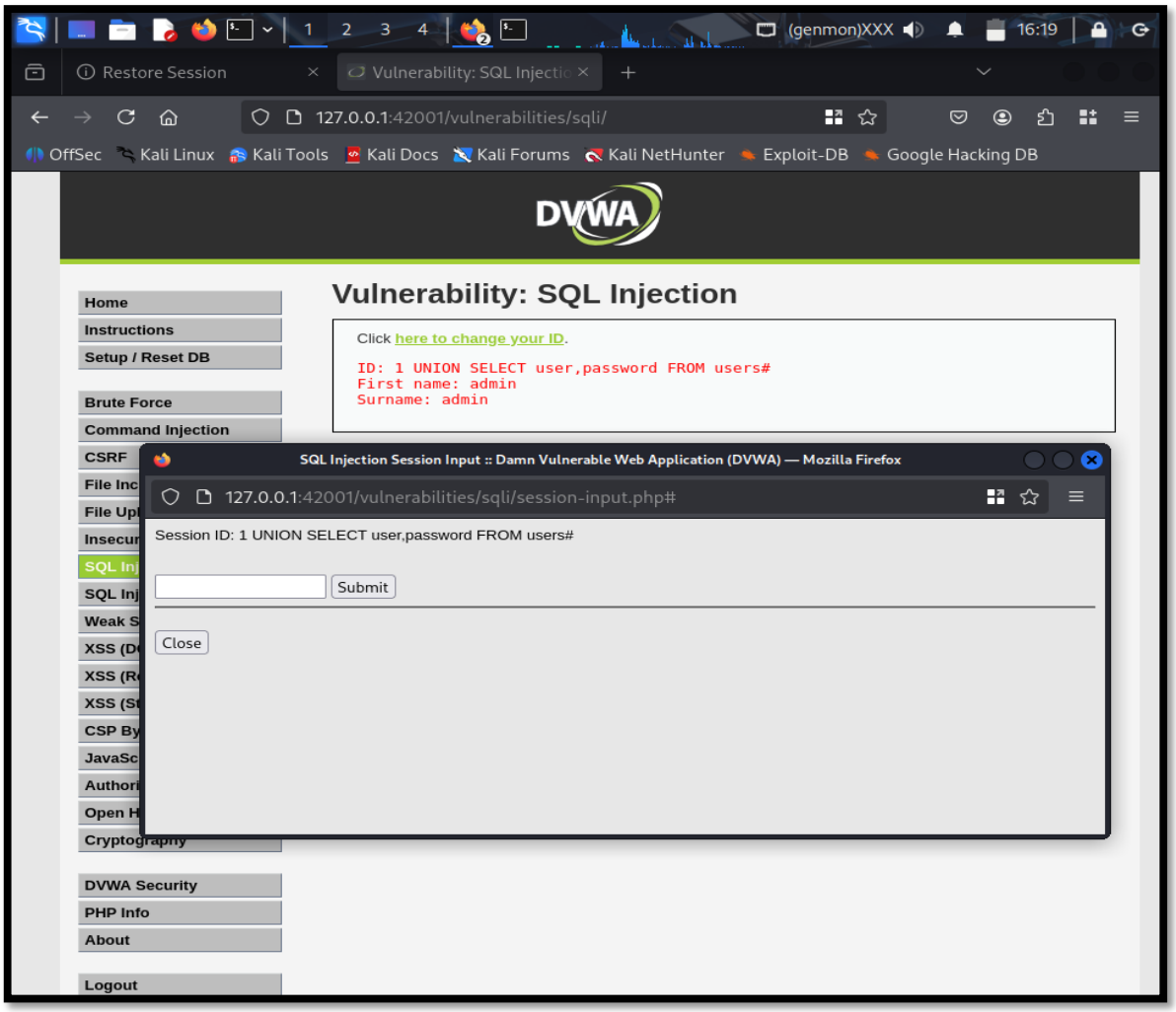
- **Screenshot 6:** Captures input, Burp Suite intercepting modified requests, and database leak result.
  Description: Demonstrates increased but incomplete security controls.

High Security

- Advanced filtering or parameterized queries block simple SQLi attempts.
- Payloads fail to produce unauthorized data; error messages suppressed to avoid information leakage.



Screenshot 7: Security Level set to High

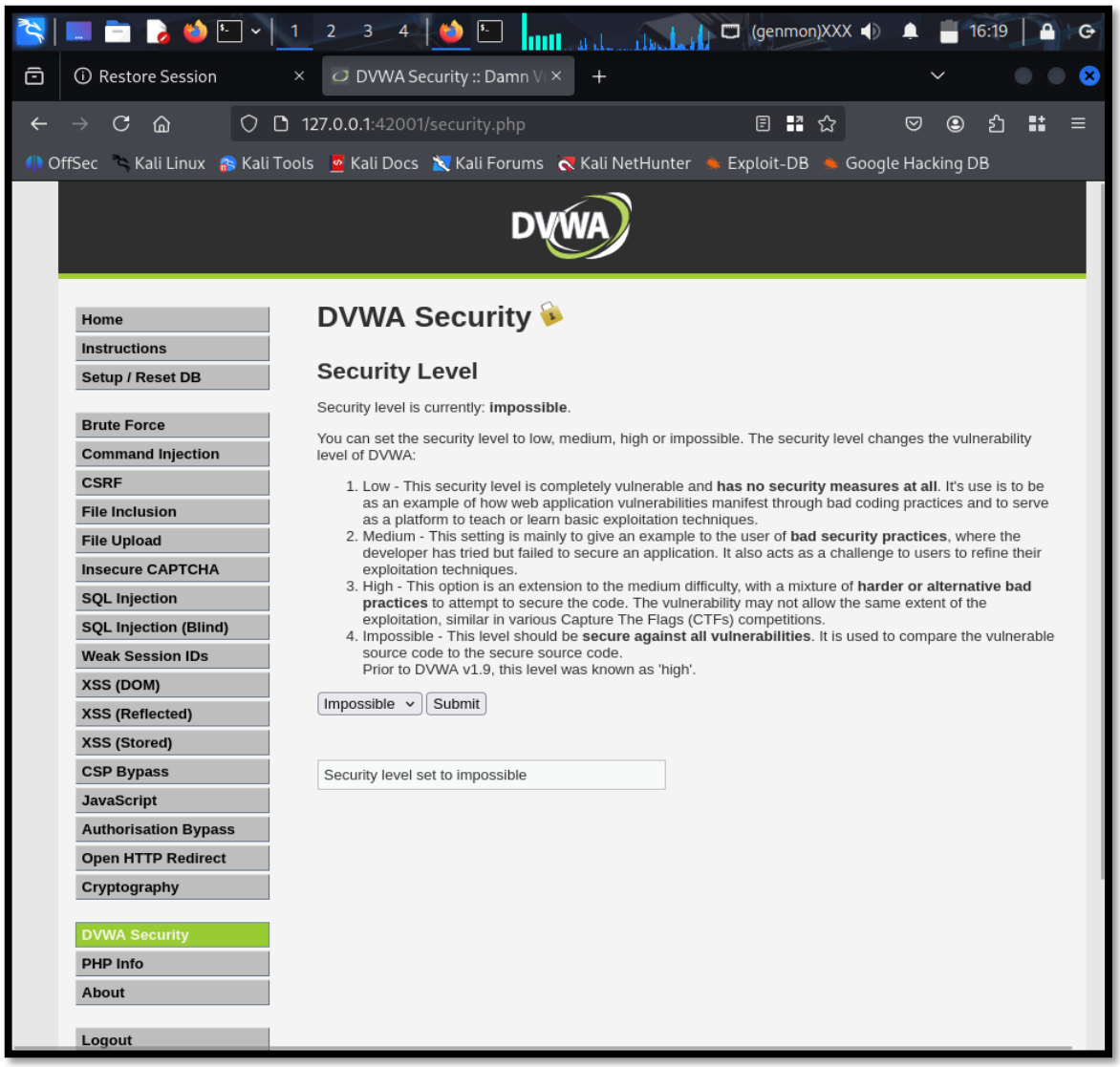


- **Screenshot 8:** Input attempt and server sanitized response or generic error page.

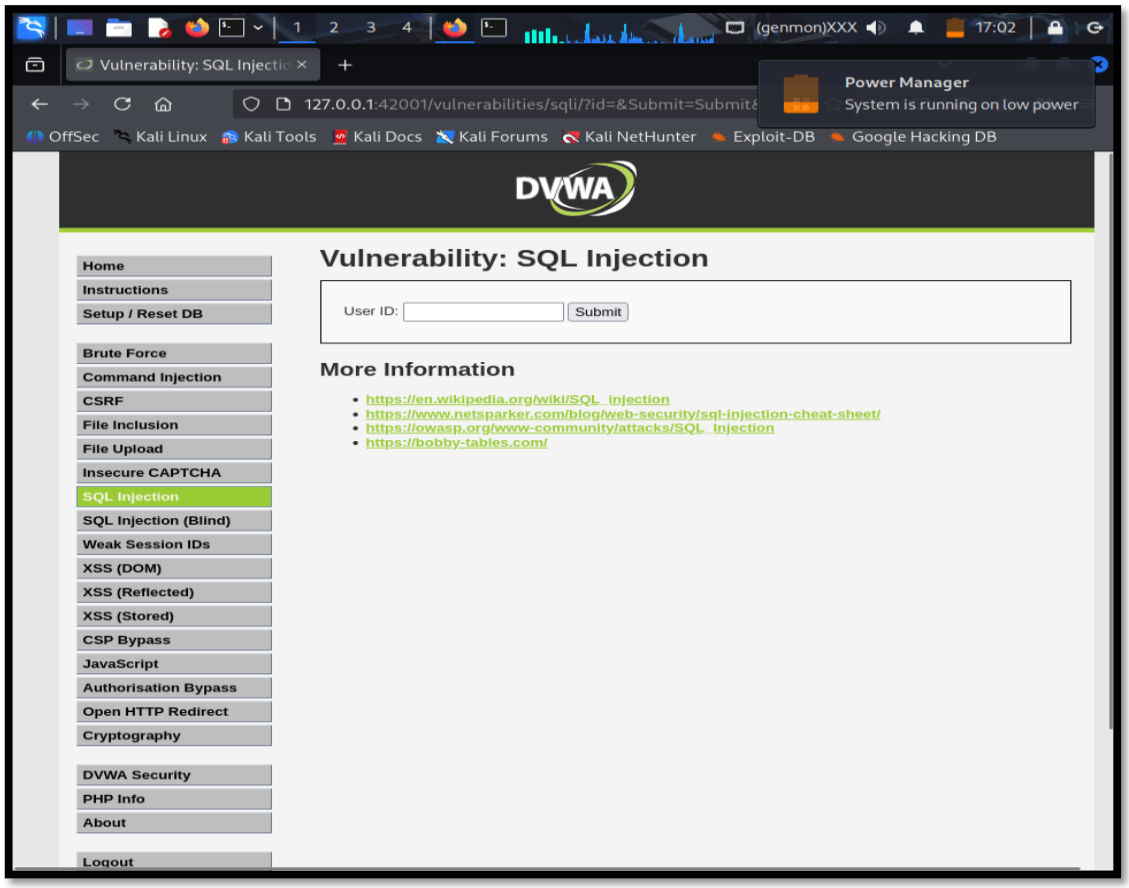
  Description: Effective mitigation of attack vectors with improved application logic.

Impossible Security

- All SQL injection attempts produce no error, no unauthorized access, or data exposure.
- Application uses fully parameterized queries and input validation preventing SQLi.



Screenshot 9: Security Level set to Impossible



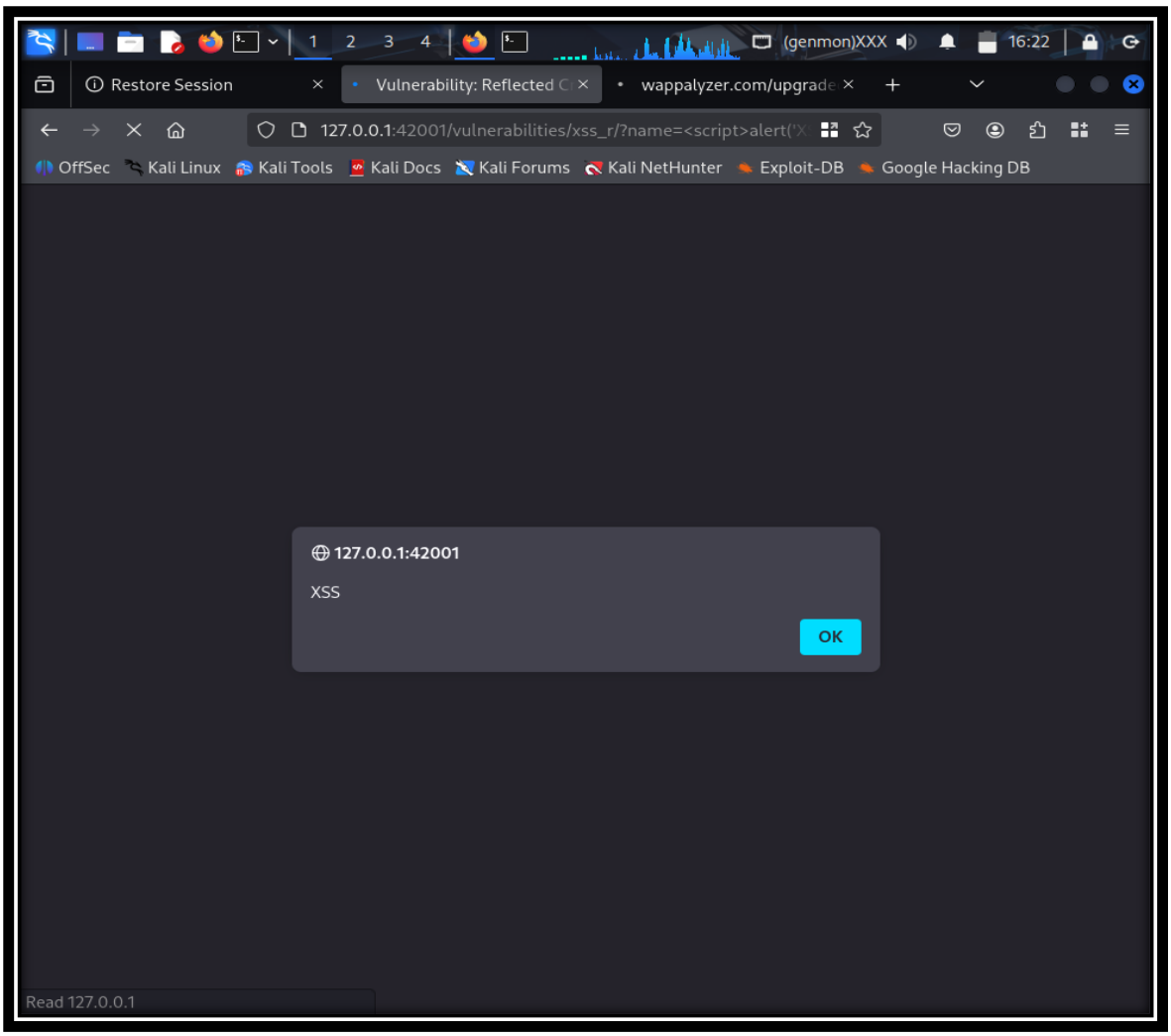
- **Screenshot 10:** Payload input form with no visible vulnerability and safe output.
  Description: Represents secure coding practices where injection vulnerabilities are eliminated.

4.2 Reflected Cross-Site Scripting (XSS)

XSS testing involves injecting JavaScript payloads in inputs which the web page reflects back, executing scripts in the user's browser.

Low Security

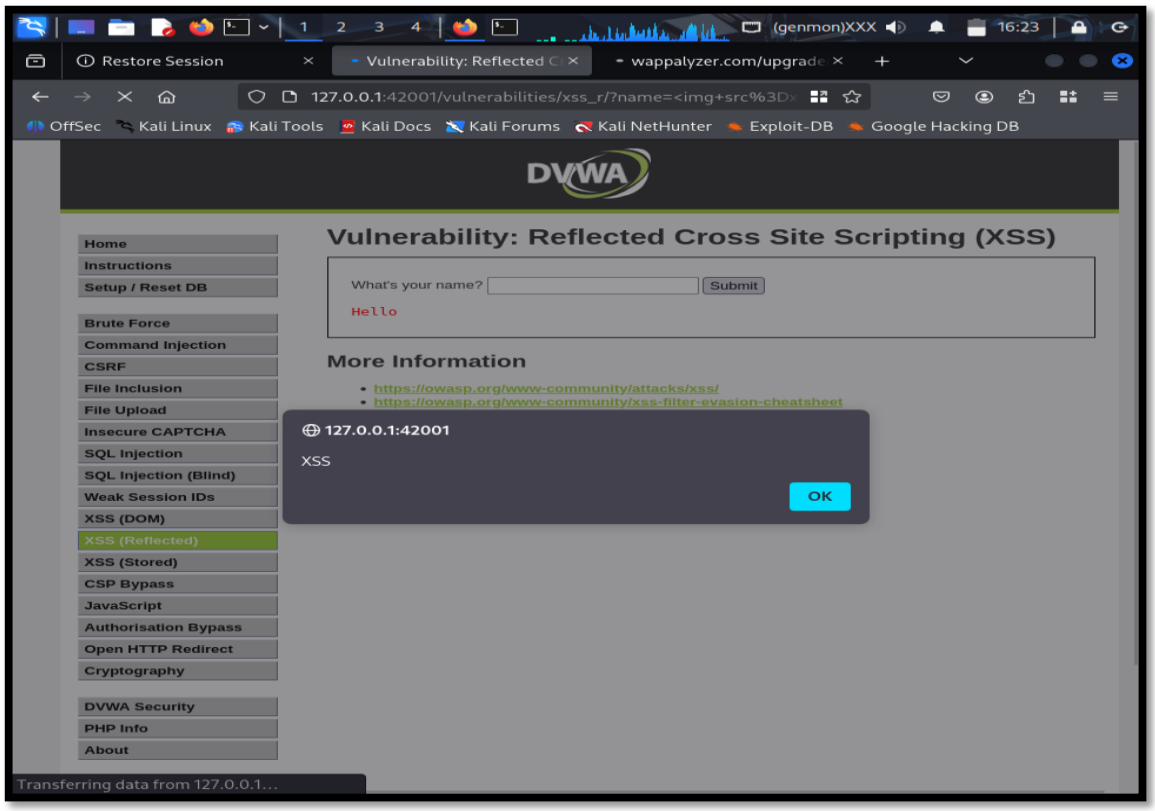
- **Payload:**
  `<script>alert('XSS')</script>`
- **Outcome:** Payload executes, popping an alert box, confirming no filtering or output encoding.

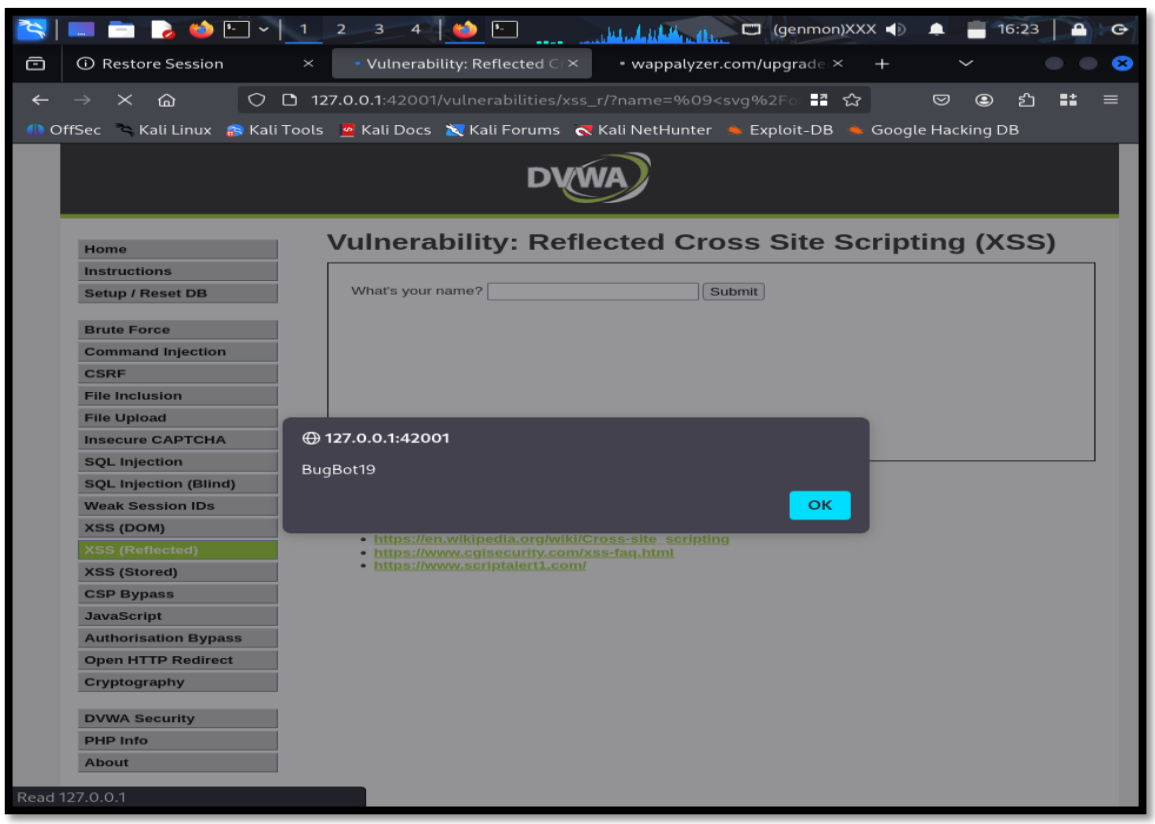


- **Screenshot 11:** Input form with payload and resultant alert popup on page reload.
  Description: Confirms vulnerable reflected XSS condition.

Medium and High Security

- Payloads such as `<img src=x onerror=alert('XSS')>` are used to bypass partial filters.
- Higher security levels sanitize or encode some inputs but occasionally allow complex scripts.

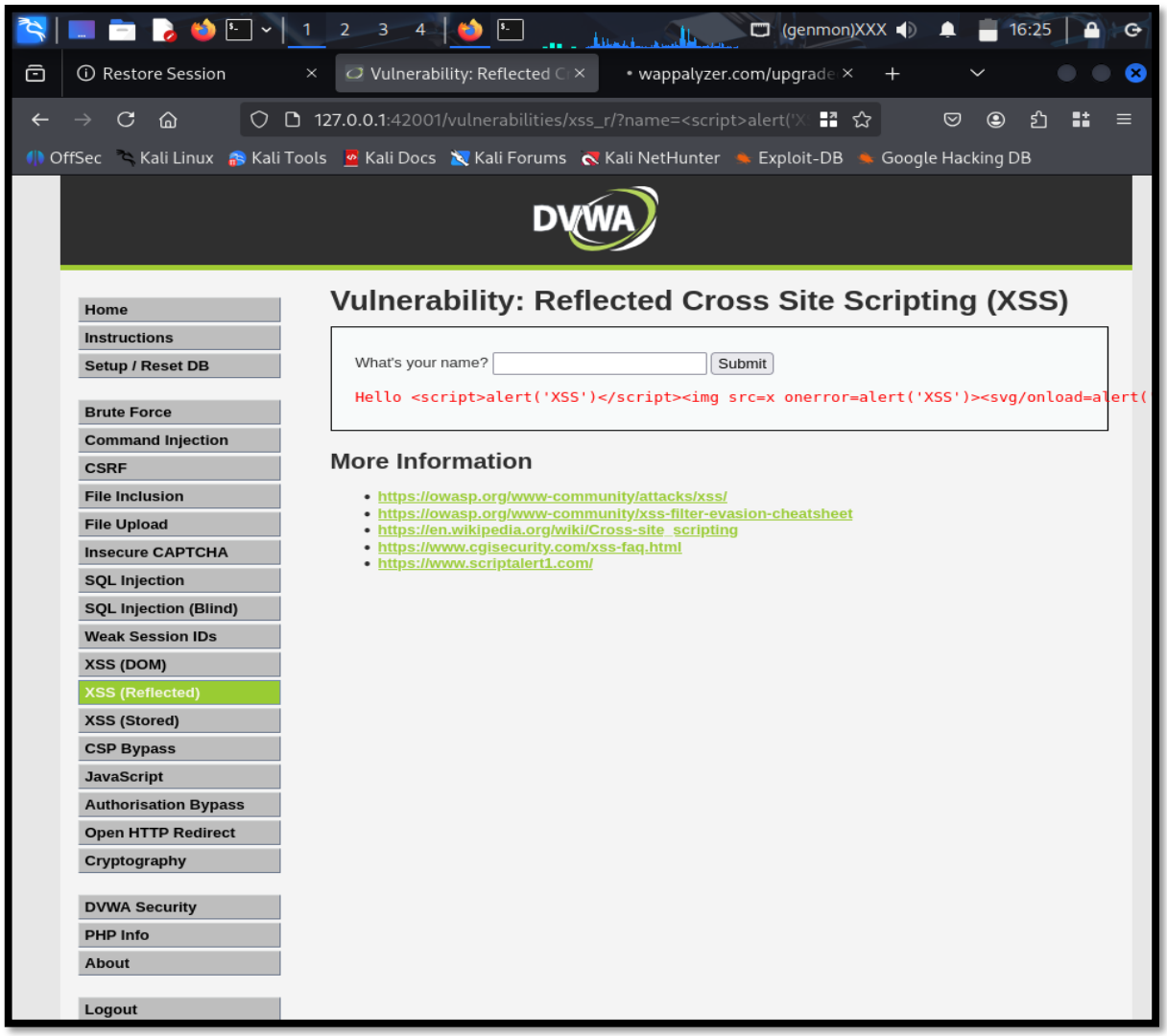

Restore Session
Vulnerability: Reflected C
wappalyzer.com/upgrade
127.0.0.1:42001/vulnerabilities/xss_r/?name=<img+src%3D
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Authorisation Bypass
Open HTTP Redirect
Cryptography
DVWA Security
PHP Info
About
Vulnerability: Reflected Cross Site Scripting (XSS)
What's your name?
Submit
Hello
More Information
https://owasp.org/www-community/attacks/xss/
https://owasp.org/www-community/xss-filter-evasion-cheatsheet
127.0.0.1:42001
XSS
OK
Transferring data from 127.0.0.1...


Restore Session
Vulnerability: Reflected C
wappalyzer.com/upgrade
127.0.0.1:42001/vulnerabilities/xss_r/?name=%09<svg%2F
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Authorisation Bypass
Open HTTP Redirect
Cryptography
DVWA Security
PHP Info
About
Vulnerability: Reflected Cross Site Scripting (XSS)
What's your name?
Submit
127.0.0.1:42001
BugBot19
OK
https://en.wikipedia.org/wiki/Cross-site_scripting
https://www.cgisecurity.com/xss-faq.html
https://www.scriptalert1.com/
Read 127.0.0.1

- **Screenshot 12 and 13:** Shows input form, attempted payload, and either successful alert or evidence of filtering (no alert).
  Description: Demonstrates implemented defense mechanisms and their limitations.

Impossible Security

- All script injection attempts are neutralized. Payloads displayed as text or removed entirely—no JavaScript execution.
- Robust Context Sensitive Output Encoding and Content Security Policies prevent script execution.



- **Screenshot 14:** Input form and clean output proving no XSS vulnerability.
  Description: Strongest defense level preventing reflected XSS attacks.

5. Analysis and Interpretation

- The Low security level offers no defenses, replicating poorly coded web applications exposed to easy exploitation.
- Medium security imitates typical flawed defenses via basic input validation but lacks full parameterized queries or encoding.
- High level introduces sophisticated mitigations like prepared SQL statements and partial filtering.
- Impossible level exemplifies best practices by eliminating injection flaws through comprehensive input validation, prepared statements for SQL, and strict output encoding for XSS.

This layered defense approach mirrors real-world application hardening, showing how each incremental step reduces attack surfaces and risk.

6. Conclusion

This project illuminates the critical importance of secure development practices in web applications. By methodically testing vulnerabilities at increasing security levels, one understands how defensive programming concepts protect sensitive data and users from malicious exploits.

Educationally, DVWA provides a safe environment to experience attacks and defenses, laying a foundation for mastering application security essential for cybersecurity professionals.

⁂